

INFS3200 A2

Alex Donnellan (46963037)

Contents

1. Task 1	2
2. Q1	2
2.1. Q2	3
2.2. Q3	4
3. Task2	5
4. Q1	5
4.1. Q2	8
4.2. Q3	9
4.3. Q4	9
4.4. c	9
4.5. Q5	10

1. Task 1

2. Q1

```
from a2.src import P4
from a2.src.helpers import nested_loop
from p4.DataLinkage_py.src.data.db_loader import db_loader
from p4.DataLinkage_py.src.data.measurement import calc_measure, load_benchmark

# REF: https://en.wikibooks.org/wiki/Algorithm_Implementation/Strings/Levenshtein_distance#Python
def levenshtein(s1, s2):
    if len(s1) < len(s2):
        return levenshtein(s2, s1)

    # len(s1) >= len(s2)
    if len(s2) == 0:
        return len(s1)

    previous_row = range(len(s2) + 1)
    for i, c1 in enumerate(s1):
        current_row = [i + 1]
        for j, c2 in enumerate(s2):
            insertions = (
                previous_row[j + 1] + 1
            ) # j+1 instead of j since previous_row and current_row are one character longer
            deletions = current_row[j] + 1 # than s2
            substitutions = previous_row[j] + (c1 != c2)
            current_row.append(min(insertions, deletions, substitutions))
        previous_row = current_row

    return previous_row[-1]

if __name__ == "__main__":
    restaurants = db_loader()
    benchmark = load_benchmark(P4 / "data" / "restaurant_pair.csv")
    for t in [0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]:
        count = 0
        res = []
        for i, j in nested_loop(restaurants):
            name1 = i.get_name()
            name2 = j.get_name()
            edi_dist = levenshtein(name1, name2)
            sim = 1 - (edi_dist / max(len(name1), len(name2)))
            if sim >= t:
                count += 1
                res.append(f"{str(i.get_id())}_{str(j.get_id())}")

    print(f"Threshold: {t} similar count: {count}")
    calc_measure(res, benchmark)
    print("-----")
```

```

s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task1.q1
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, 64-bit',)
Threshold: 0.4 similar count: 4142
Precision= 0.022935779816513763 , Recall= 0.8962264150943396 , Fmeasure= 0.04472693032015067
-----
Threshold: 0.5 similar count: 1349
Precision= 0.06523350630096368 , Recall= 0.8301886792452831 , Fmeasure= 0.12096219931271479
-----
Threshold: 0.6 similar count: 359
Precision= 0.23119777158774374 , Recall= 0.7830188679245284 , Fmeasure= 0.35698924731182796
-----
Threshold: 0.7 similar count: 139
Precision= 0.5683453237410072 , Recall= 0.7452830188679245 , Fmeasure= 0.6448979591836734
-----
Threshold: 0.8 similar count: 97
Precision= 0.8144329896907216 , Recall= 0.7452830188679245 , Fmeasure= 0.7783251231527093
-----
Threshold: 0.9 similar count: 86
Precision= 0.8953488372093024 , Recall= 0.7264150943396226 , Fmeasure= 0.8020833333333334
-----
Threshold: 1.0 similar count: 82
Precision= 0.9146341463414634 , Recall= 0.7075471698113207 , Fmeasure= 0.7978723404255319
-----
s4696303@infs3200-0b490d8e:~/infs3200$ 

```

2.1. Q2

```

from a2.src import P4
from a2.src.helpers import nested_loop
from p4.DataLinkage_py.src.data.db_loader import db_loader
from p4.DataLinkage_py.src.data.measurement import calc_measure, load_benchmark

def qgram(str: str, q: int) -> set[str]:
    return {str[i : i + 1] for i in range(len(str) - q + 1)}

def jaccard(qgram1: set, qgram2: set):
    if not qgram1 or not qgram2:
        return 1.0
    return len(qgram1 & qgram2) / len(qgram1 | qgram2)

if __name__ == "__main__":
    restaurants = db_loader()
    benchmark = load_benchmark(P4 / "data" / "restaurant_pair.csv")
    for q in [2, 3, 4]:
        for t in [0.6, 0.7, 0.8, 0.9, 1.0]:
            count = 0
            res = []
            for i, j in nested_loop(restaurants):
                name1 = i.get_name()
                name2 = j.get_name()
                sim = jaccard(qgram(name1, q), qgram(name2, q))
                if sim >= t:
                    count += 1
                    res.append(f"{str(i.get_id())}_{str(j.get_id())}")

            print(f"Q: {q} Threshold: {t} similar count: {count}")
            calc_measure(res, benchmark)
            print("-----")

```

```

s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task1.q2
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 1
3.3.0, 64-bit',)
Q: 2 Threshold: 0.6 similar count: 7172
Precision= 0.01324595649749024 , Recall= 0.8962264150943396 , Fmeasure= 0.026106073097004672
-----
Q: 2 Threshold: 0.7 similar count: 1270
Precision= 0.07086614173228346 , Recall= 0.8490566037735849 , Fmeasure= 0.1308139534883721
-----
Q: 2 Threshold: 0.8 similar count: 259
Precision= 0.3281853281853282 , Recall= 0.8018867924528302 , Fmeasure= 0.4657534246575343
-----
Q: 2 Threshold: 0.9 similar count: 103
Precision= 0.7864077669902912 , Recall= 0.7641509433962265 , Fmeasure= 0.7751196172248804
-----
Q: 2 Threshold: 1.0 similar count: 92
Precision= 0.8586956521739131 , Recall= 0.7452830188679245 , Fmeasure= 0.7979797979797979
-----
Q: 3 Threshold: 0.6 similar count: 7438
Precision= 0.012368916375369724 , Recall= 0.8679245283018868 , Fmeasure= 0.024390243902439025
-----
Q: 3 Threshold: 0.7 similar count: 2737
Precision= 0.03251735476799415 , Recall= 0.839622641509434 , Fmeasure= 0.06260991909954274
-----
Q: 3 Threshold: 0.8 similar count: 1943
Precision= 0.04323211528564076 , Recall= 0.7924528301886793 , Fmeasure= 0.08199121522693997
-----
Q: 3 Threshold: 0.9 similar count: 1820
Precision= 0.04450549450549451 , Recall= 0.7641509433962265 , Fmeasure= 0.08411214953271028
-----
Q: 3 Threshold: 1.0 similar count: 1811
Precision= 0.04251794588625069 , Recall= 0.7264150943396226 , Fmeasure= 0.08033385498174231
-----
Q: 4 Threshold: 0.6 similar count: 9912
Precision= 0.009180790960451978 , Recall= 0.8584905660377359 , Fmeasure= 0.018167298862048313
-----
Q: 4 Threshold: 0.7 similar count: 6047
Precision= 0.0143872994873491 , Recall= 0.8207547169811321 , Fmeasure= 0.028278888347147736
-----
Q: 4 Threshold: 0.8 similar count: 5391
Precision= 0.015396030421072158 , Recall= 0.7830188679245284 , Fmeasure= 0.030198289976350736
-----
Q: 4 Threshold: 0.9 similar count: 5268
Precision= 0.015186028853454821 , Recall= 0.7547169811320755 , Fmeasure= 0.0297729810197246
-----
Q: 4 Threshold: 1.0 similar count: 5263
Precision= 0.01444043321299639 , Recall= 0.7169811320754716 , Fmeasure= 0.02831067237846899
-----

```

2.2. Q3

```

from a2.src import P4
from a2.src.helpers import nested_loop
from a2.src.task1.q2 import jaccard, qgram
from p4.DataLinkage_py.src.data.db_loader import db_loader
from p4.DataLinkage_py.src.data.measurement import calc_measure, load_benchmark

```

```

if __name__ == "__main__":
    restaurants = db_loader()
    benchmark = load_benchmark(P4 / "data" / "restaurant_pair.csv")
    q = 3
    max_f_measure = 0
    max_a = 0
    for a in [0.5, 0.7, 0.9]:
        for t in [0.5, 0.7, 0.9]:
            count = 0
            res = []
            for i, j in nested_loop(restaurants):
                name1 = i.get_name()

```

```

addr1 = i.get_address()
name2 = j.get_name()
addr2 = j.get_address()
sim_name = jaccard(qgram(name1, q), qgram(name2, q))
sim_addr = jaccard(qgram(addr1, q), qgram(addr2, q))
sim = a * sim_name + (1 - a) * sim_addr
if sim >= t:
    count += 1
    res.append(f"{str(i.get_id())}_{str(j.get_id())}")

print(f"Q: {q} Threshold: {t} similar count: {count}")
_, _, f_measure = calc_measure(res, benchmark)
print("-----")
if f_measure > max_f_measure:
    max_f_measure = f_measure
    max_a = a
print(f"Max pair ({max_a}, {max_f_measure})")

```

```

s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task1.q3
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, 64-bit',)
Q: 3 Threshold: 0.5 similar count: 8667
Precision= 0.011999538479289258 , Recall= 0.9811320754716981 , Fmeasure= 0.02370910748888636
-----
Q: 3 Threshold: 0.7 similar count: 1208
Precision= 0.0802980132450331 , Recall= 0.9150943396226415 , Fmeasure= 0.1476407914764079
-----
Q: 3 Threshold: 0.9 similar count: 64
Precision= 0.90625 , Recall= 0.5471698113207547 , Fmeasure= 0.6823529411764705
-----
Q: 3 Threshold: 0.5 similar count: 11012
Precision= 0.009081002542680712 , Recall= 0.9433962264150944 , Fmeasure= 0.017988846914912753
-----
Q: 3 Threshold: 0.7 similar count: 1934
Precision= 0.047569803516028956 , Recall= 0.8679245283018868 , Fmeasure= 0.09019607843137256
-----
Q: 3 Threshold: 0.9 similar count: 71
Precision= 0.8732394366197183 , Recall= 0.5849056603773585 , Fmeasure= 0.7005649717514125
-----
Q: 3 Threshold: 0.5 similar count: 18353
Precision= 0.005339726475235656 , Recall= 0.9245283018867925 , Fmeasure= 0.010618126659082289
-----
Q: 3 Threshold: 0.7 similar count: 2224
Precision= 0.04091726618705036 , Recall= 0.8584905660377359 , Fmeasure= 0.07811158798283262
-----
Q: 3 Threshold: 0.9 similar count: 1815
Precision= 0.04462809917355372 , Recall= 0.7641509433962265 , Fmeasure= 0.08433107756376887
-----
Max pair (0.7, 0.7005649717514125)
s4696303@infs3200-0b490d8e:~/infs3200$

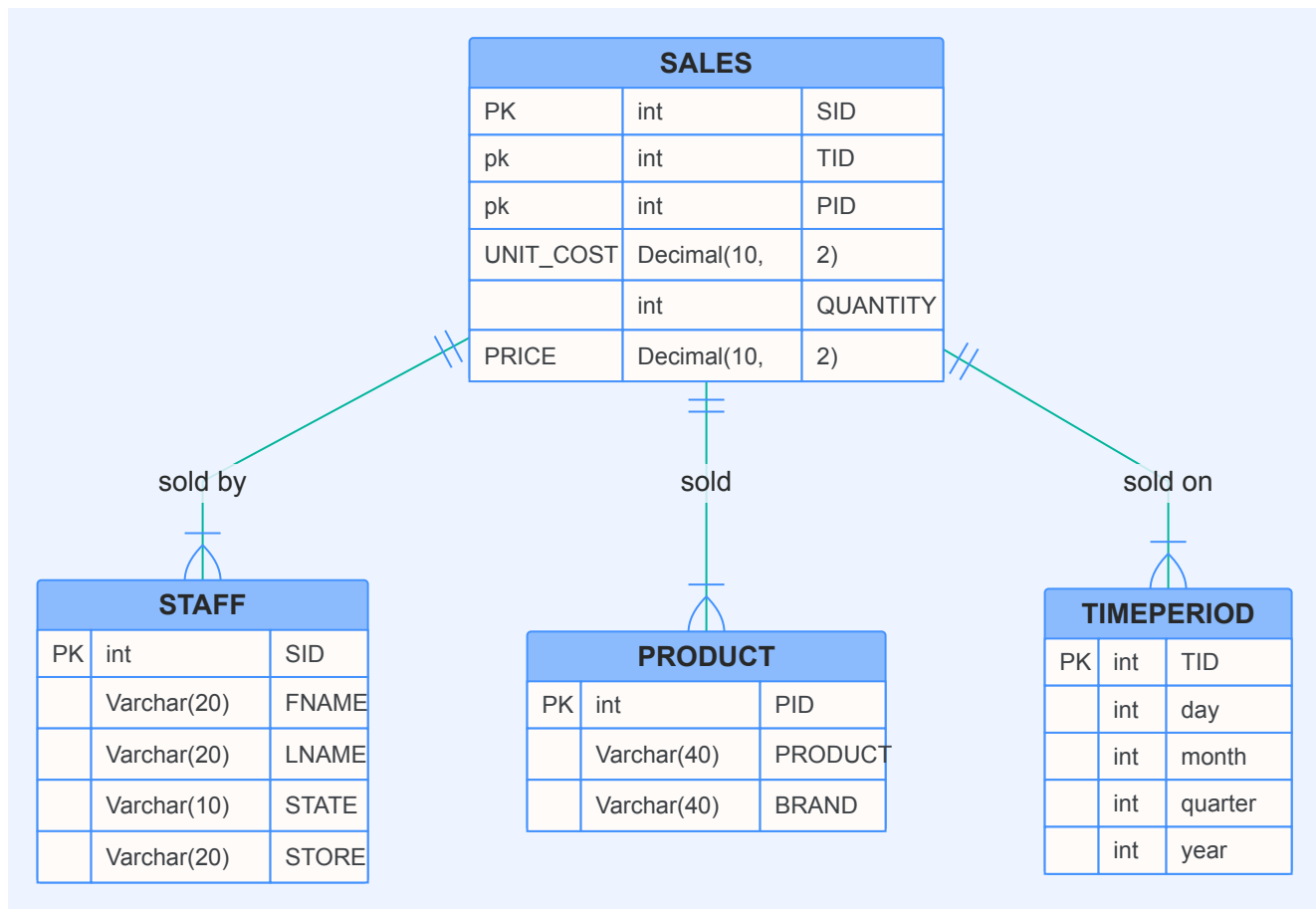
```

3. Task2

4. Q1

First we build out the star schema

Alex Donnellan (46963037)



then we create this in postgres

```
CREATE DATABASE "A2";
\c "A2"
CREATE USER "a2" WITH PASSWORD 'infs3200';
GRANT ALL PRIVILEGES ON DATABASE "A2" TO "a2";
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL ON TABLES TO "a2";
CREATE TABLE staff (sid int PRIMARY KEY, fname VARCHAR(20), lname VARCHAR(20), state VARCHAR(10), store VARCHAR(20) );
CREATE TABLE timeperiod (tid int PRIMARY KEY, day int, month int, quarter int, year int);
CREATE TABLE product (pid int PRIMARY KEY, product VARCHAR(40), brand VARCHAR(40));
CREATE TABLE SALES (
    sid int,
    tid int,
    pid int,
    unit_cost Decimal(10, 2),
    quantity int,
    price Decimal(10, 2),
    PRIMARY KEY (sid, tid, pid),
    CONSTRAINT fk_staff FOREIGN KEY (sid) REFERENCES staff(sid),
    CONSTRAINT fk_timeperiod FOREIGN KEY (tid) REFERENCES timeperiod(tid),
    CONSTRAINT fk_product FOREIGN KEY (pid) REFERENCES product(pid)
);
```

```

s4696303@infs3200-0b490d8e:~$ psql
psql (17.7 (Ubuntu 17.7-3.pgdg24.04+1))
Type "help" for help.

s4696303=# CREATE DATABASE "A2";
CREATE DATABASE
s4696303=# \c "A2";
You are now connected to database "A2" as user "s4696303".
A2=# CREATE USER "a2" WITH PASSWORD 'infs3200';
CREATE ROLE
A2=# GRANT ALL PRIVILEGES ON DATABASE "A2" TO "a2";
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL ON TABLES TO "a2";
GRANT
ALTER DEFAULT PRIVILEGES
A2=# CREATE TABLE staff (id SERIAL PRIMARY KEY, fname VARCHAR(20), lname VARCHAR(20), state VARCHAR(10), store VARCHAR(20) );
CREATE TABLE timeperiod (id SERIAL PRIMARY KEY, day int, month int, quarter int, year int);
CREATE TABLE product (id SERIAL PRIMARY KEY, product VARCHAR(40), brand VARCHAR(40));
CREATE TABLE SALES (
    sid int,
    tid int,
    pid int,
    unit_cost Decimal(10, 2),
    quantity int,
    price Decimal(10, 2),
    PRIMARY KEY (sid, tid, pid),
    CONSTRAINT fk_staff FOREIGN KEY (sid) REFERENCES staff(id),
    CONSTRAINT fk_timeperiod FOREIGN KEY (tid) REFERENCES timeperiod(id),
    CONSTRAINT fk_product FOREIGN KEY (pid) REFERENCES product(id)
)
CREATE TABLE
CREATE TABLE
CREATE TABLE
A2=# ;
CREATE TABLE
A2=#

```

Next we need to load data in we'll do that with the following script

```

from datetime import datetime
from math import ceil
from pathlib import Path

from p4.DataLinkage_py.src.psql.DBconnect import create_connection

ROOT = Path(__file__).parent

if __name__ == "__main__":
    conn = create_connection(database="A2", user="a2")
    count = 0
    with conn.cursor() as cur:
        cur.execute("TRUNCATE TABLE staff, timeperiod, product, sales;")
        with open(ROOT / "Sales.csv", "r") as f:
            # Skip the first line
            f.readline()
            while True:
                line = f.readline()
                if not line:
                    break
                line = line.replace("'", " ")
                line = line.split(",")
                # TID,SID,FNAME,LNAME,STATE,STORE,DATE,PID,BRAND,PRODUCT,UNIT_COST,QUANTITY,PRICE
                (
                    tid,
                    sid,
                    fname,
                    lname,
                    state,
                    store,
                    date,

```



```

        pid,
        brand,
        product,
        unit_cost,
        quantity,
        price,
    ) = line
    # Make the bold assumption that the data is clean and accept only the first copy
    cur.execute(
        """INSERT INTO staff (sid, fname, lname, state, store)
        VALUES (%s, %s, %s, %s, %s) ON CONFLICT (sid) DO NOTHING;""",
        (sid, fname, lname, state, store),
    )
    sale_date = datetime.fromisoformat(date)
    cur.execute(
        """INSERT INTO timeperiod (tid, day, month, quarter, year) VALUES (%s, %s, %s, %s, %s) ON
CONFLICT (tid) DO NOTHING""",
        (
            tid,
            sale_date.day,
            sale_date.month,
            ceil(sale_date.month / 3),
            sale_date.year,
        ),
    )
    cur.execute(
        """INSERT INTO product (pid, product, brand) VALUES (%s, %s, %s) ON CONFLICT (pid) DO
NOTHING""",
        (pid, brand, product),
    )
    cur.execute(
        """INSERT INTO sales (sid, tid, pid, unit_cost, quantity, price) VALUES (%s, %s, %s, %s, %s,
%s)""",
        (sid, tid, pid, unit_cost, quantity, price),
    )
    count += 1
    conn.commit()
    print(f"Added {count} rows")

```

```

s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task2.q1
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04
) 13.3.0, 64-bit',)
Added 234680 rows
s4696303@infs3200-0b490d8e:~/infs3200$ 

```

4.1. Q2

4.1.1. a

```
SELECT COUNT(*) FROM staff;
```

```

s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task2.q1
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04
) 13.3.0, 64-bit',)
Added 234680 rows
s4696303@infs3200-0b490d8e:~/infs3200$ 

```

Because the insert script doesn't allow duplicates we can just count the whole table

4.1.2. b

```
SELECT COUNT(*) FROM sales s INNER JOIN timeperiod t ON t.tid = s.tid WHERE t.year = 2022 AND t.quarter = 3;
```

```
s4696303@infs3200-0b490d8e:~/infs3200$ uv run python -m a2.src.task2.q1
PostgreSQL database version:
('PostgreSQL 17.7 (Ubuntu 17.7-3.pgdg24.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, 64-bit',)
Added 234680 rows
s4696303@infs3200-0b490d8e:~/infs3200$
```

4.2. Q3

```
CREATE MATERIALIZED VIEW "Sales_Time_Staff" AS
SELECT s.sid as sid, s.state as state, s.store as store, t.day as day, t.month as month, t.quarter as quarter,
t.year as year,
SUM(fs.quantity * fs.price) as total_revenu, SUM(fs.quantity * fs.unit_cost) as total_cost, SUM(fs.quantity *
(fs.price - fs.unit_cost)) as total_profit, SUM(fs.quantity) as total_sold
FROM sales as fs
INNER JOIN staff s ON fs.sid = s.sid
INNER JOIN timeperiod t ON fs.tid = t.tid
GROUP BY ROLLUP (s.state, s.store, s.sid), ROLLUP (t.year, t.quarter, t.month, t.day);
```

```
A2=# CREATE MATERIALIZED VIEW "Sales_Time_Staff" AS
SELECT s.sid as sid, s.state as state, s.store as store, t.day as day, t.month as month, t.quarter as quarter, t.year as year,
SUM(fs.quantity * fs.price) as total_revenu, SUM(fs.quantity * fs.unit_cost) as total_cost, SUM(fs.quantity * (fs.price - fs.unit_cost)) as total_profit, SUM(fs.quantity) as total_sold
FROM sales as fs
INNER JOIN staff s ON fs.sid = s.sid
INNER JOIN timeperiod t ON fs.tid = t.tid
GROUP BY ROLLUP (s.state, s.store, s.sid), ROLLUP (t.year, t.quarter, t.month, t.day);
SELECT 189645
A2=#
```

4.3. Q4

4.3.1. a

```
SELECT total_profit FROM "Sales_Time_Staff" WHERE year = 2021 AND quarter = 4 AND day IS NULL AND month IS NULL and
sid IS NULL and state IS NULL and store IS NULL;
```

```
A2=# SELECT total_profit FROM "Sales_Time_Staff" WHERE year = 2021 AND quarter = 4 AND day IS NULL AND month IS NULL and sid IS NULL and state IS NULL and store IS NULL;
total_profit
-----
88530899.83
(1 row)
A2=#
```

4.3.2. b

```
SELECT year, state, total_profit FROM "Sales_Time_Staff" WHERE year IN (2021, 2022) AND state IN ('NSW', 'QLD') AND
quarter IS NULL AND day IS NULL AND month IS NULL and sid IS NULL and store IS NULL;
```

```
A2=# SELECT year, state, total_profit FROM "Sales_Time_Staff" WHERE year IN (2021, 2022) AND state IN ('NSW', 'QLD') AND quarter IS NULL AND day IS NULL AND month IS NULL and sid IS NULL and store IS NULL;
year | state | total_profit
-----+-----+-----
2021 | NSW   | 103711376.14
2021 | QLD   | 84291709.15
2022 | NSW   | 101622087.94
2022 | QLD   | 83071386.73
(4 rows)
A2=#
```

4.4. c

```
SELECT * FROM crosstab('SELECT store, quarter, total_profit FROM "Sales_Time_Staff"
WHERE year = 2021 AND day IS NULL AND month IS NULL AND sid IS NULL AND quarter IS NOT NULL AND
store IN (SELECT store FROM "Sales_Time_Staff" WHERE year = 2021 AND day IS NULL AND month IS NULL AND sid IS NULL
AND quarter IS NULL AND store IS NOT NULL
ORDER BY total_profit DESC LIMIT 3
)', 'SELECT DISTINCT quarter FROM timeperiod') as final_result (store TEXT, q1 NUMERIC, q2 NUMERIC, q3 NUMERIC, q4
NUMERIC);
```

```
A2=# SELECT * FROM crosstab('SELECT store, quarter, total_profit FROM "Sales_Time_Staff" WHERE year = 2021 AND day IS NULL AND month IS NULL AND sid IS NULL AND quarter IS NOT NULL AND store IN (SELECT store FROM "Sales_Time_Staff" WHERE year = 2021 AND day IS NULL AND month IS NULL AND sid IS NULL AND quarter IS NOT NULL ORDER BY total_profit DESC LIMIT 3)', 'SELECT DISTINCT quarter FROM timeperiod') as final_result (store TEXT, q1 NUMERIC, q2 NUMERIC, q3 NUMERIC, q4 NUMERIC);
store | q1 | q2 | q3 | q4
-----+-----+-----+-----+-----
S02   | 7886126.92 | 8411775.83 | 7775546.81 | 7889620.53
W01   | 8581243.98 | 8815872.09 | 8652051.61 | 8698293.25
W02   | 11738990.73 | 11938869.03 | 11855089.43 | 12331896.25
(3 rows)
```

A2=#

4.5. Q5

```
CREATE MATERIALIZED VIEW "Sales_Product_Staff" AS
SELECT s.sid as sid, s.state as state, s.store as store, p.product as product, p.brand as brand, SUM(fs.quantity *
fs.unit_cost) as total_cost, SUM(fs.quantity * (fs.price - fs.unit_cost)) as total_profit, SUM(fs.quantity) as
total_sold
FROM sales as fs
INNER JOIN staff s ON fs.sid = s.sid
INNER JOIN product p on fs.pid = p.pid
GROUP BY ROLLUP(s.state, s.store, s.sid), ROLLUP(p.brand, p.product);
```

```
A2=# CREATE MATERIALIZED VIEW "Sales_Product_Staff" AS
SELECT s.sid as sid, s.state as state, s.store as store, p.product as product, p.brand as brand, SUM(fs.quantity * fs.unit_cost) as total_cost, SUM(fs.quantity * (fs.price - fs.unit_cost)) as total_profit, SUM(fs.quantity) as total_sold
FROM sales as fs
INNER JOIN staff s ON fs.sid = s.sid
INNER JOIN product p on fs.pid = p.pid
GROUP BY ROLLUP(s.state, s.store, s.sid), ROLLUP(p.brand, p.product);
SELECT 63632
A2=#
```

4.5.1. a

```
CREATE VIEW "Top_3_Stores" AS
SELECT store FROM "Sales_Product_Staff" WHERE product IS NULL AND brand IS NULL AND store IS NOT NULL ORDER BY
total_profit DESC LIMIT 3;
```

```
A2=# CREATE VIEW "Top_3_Stores" AS
SELECT store FROM "Sales_Product_Staff" WHERE product IS NULL AND brand IS NULL AND store IS NOT NULL ORDER BY total_profit DESC LIMIT 3;
CREATE VIEW
A2=# SELECT * FROM "Top_3_Stores";
store
-----
W02
W01
S02
(3 rows)
```

A2=#

4.5.2. b

```
CREATE VIEW "Top_Products" AS
SELECT DISTINCT ON (store) store, product, total_profit FROM "Sales_Product_Staff" WHERE store IS NOT NULL AND
product IS NOT NULL ORDER BY store, total_profit DESC;
```

```
A2=# CREATE VIEW "Top_Products" AS
SELECT DISTINCT ON (store) store, product, total_profit FROM "Sales_Product_Staff" WHERE store IS NOT NULL AND product IS NOT NULL ORDER BY store, total_profit DESC;
CREATE VIEW
A2=# SELECT * FROM "Top_Products";
store | product | total_profit
-----+-----+-----
NSW01 | OGtech | 2038484.20
NSW02 | OGtech | 2451122.35
NSW03 | Tony | 2583373.53
NSW04 | OGtech | 2739295.50
NSW05 | OGtech | 1894512.43
NSW06 | Tony | 3185814.86
Q01 | OGtech | 3163650.98
Q02 | OGtech | 3759314.70
Q03 | OGtech | 2831220.43
Q04 | Tony | 2050304.58
S01 | OGtech | 3487603.73
S02 | OGtech | 5097017.32
S03 | OGtech | 2863287.85
W01 | OGtech | 4846956.47
W02 | OGtech | 6643037.81
(15 rows)
```

A2=#